

PaperLens

Product Requirements Document (PRD)

Version: 2.0 **Status:** Draft **Team Size:** 5 Members **Hackathon Duration:** 24 Hours

Changelog from v1.0:

- Added Feature 4B — Evidence Verification Pipeline (core differentiator, no longer just a prompt instruction)
 - Added Feature 9 — Contradiction / Consistency Checker
 - Added Feature 10 — Verified Summary Export
 - Reprioritized MVP stop points around verification instead of Family Tree / Reviewer Mode
 - Added concrete reliability/fallback requirements for external API dependencies
 - Softened NFR accuracy claim into a measurable, engineered target
-

1. Overview

Product Name

PaperLens - Evidence-Grounded Research Tutor

Vision

PaperLens transforms dense academic research papers into interactive learning experiences.

Instead of simply summarizing PDFs, PaperLens explains research while **proving every explanation using evidence directly from the uploaded paper** — and unlike most AI summarizers, it **programmatically verifies** that proof rather than trusting the LLM to cite honestly.

The primary objective is to help students understand research papers significantly faster without sacrificing factual accuracy.

2. Problem Statement

Students often struggle with research papers because they are:

- Long (20–60 pages)
- Highly technical
- Filled with unfamiliar terminology
- Difficult to navigate
- Hard to determine what is actually important

Current AI summarizers suffer from one major issue:

They summarize well, but users cannot verify whether the information actually exists in the paper — and the citations/quotes an LLM produces are frequently hallucinated.

This creates trust issues.

PaperLens solves this by grounding every generated insight back to the original PDF **and verifying that grounding programmatically**, not just asking the model to "please cite your source."

3. Goals

Primary Goal

Help students understand research papers faster while maintaining factual grounding that is engineered to be verifiable, not merely claimed.

Success Criteria

The product should enable a user to:

- Upload a research paper
 - Understand the paper in under 5 minutes
 - Navigate directly to supporting evidence
 - Ask questions about the paper
 - See, at a glance, whether each answer is actually confirmed in the source text
 - Verify every AI-generated answer
-

4. Target Users

Primary Users

- Undergraduate students
 - Graduate students
 - Researchers
 - Professors
 - Academic clubs
-

5. Core Value Proposition

"Every AI-generated insight comes with proof — and the proof is checked, not just claimed."

Unlike ordinary AI summaries, PaperLens always provides:

- Page Number
- Supporting Quote
- Source Section
- A verification badge showing whether the quote was actually found on that page

6. Scope

In Scope

✓ PDF Upload ✓ Paper Parsing ✓ Structured Summary ✓ Evidence Grounding ✓ **Evidence Verification Pipeline (new)** ✓ PDF Navigation ✓ Chat with Paper ✓ Citation Explorer ✓ Research Family Tree ✓ Reviewer Mode ✓ **Contradiction / Consistency Checker (new)** ✓ **Verified Summary Export (new)**

Out of Scope

✗ Editing PDFs ✗ Generating research papers ✗ Downloading every cited paper ✗ Multi-document comparison ✗ Citation generation ✗ User authentication

7. User Journey

Step 1

Upload a research paper. ↓

Step 2

Paper is processed and verified in the background (parsing → chunking → summarization → quote verification). ↓

Step 3

Dashboard appears showing

- Summary
- Contributions
- Methodology
- Results
- Limitations

Each claim shows a  Verified /  Paraphrased /  Unsupported badge. ↓

Step 4

Click any insight. ↓ PDF jumps directly to supporting evidence, with the matched text highlighted. ↓

Step 5

Ask questions. ↓ Receive grounded, verified answers with page references and confidence. ↓

Step 6

Explore cited papers. ↓

Step 7

Review reproducibility, weaknesses, and any internal contradictions flagged in the paper. ↓

Step 8

Export a Verified Summary Card to share or reference later.

8. Features

Feature 1 — PDF Upload

Description

Upload any research paper in PDF format.

Inputs

- PDF

Outputs

Parsed document

Feature 2 — Document Parser

Responsibilities

Extract

- Sections
- Text
- Page Numbers
- References

Output

```
json
{
  "section": "Methodology",
  "page": 6,
  "text": "..."
```

Feature 3 — Structured Summary

Generate

Main Contribution

What is the paper trying to achieve?

Methodology

How was it solved?

Results

What did they achieve?

Limitations

What are the weaknesses?

Prerequisites

Concepts needed before reading.

Each section must include evidence pages, and each claim is passed through the Evidence Verification Pipeline (Feature 4B) before being shown to the user.

Feature 4 — Evidence Grounding

The most important feature.

Every generated claim must include

- Page Number
- Supporting Quote
- Confidence

Example

```
Claim
Accuracy improved by 5%.

Evidence
Page 12

Quote
"...improved from 82.3% to 87.4%."
```

Clicking this claim opens the PDF on Page 12 with the matched span highlighted.

Feature 4B — Evidence Verification Pipeline (*new — core differentiator*)

Why this exists

LLMs frequently hallucinate quotes and page numbers even when explicitly instructed to cite. Feature 4 alone is a *prompt* promise. This feature turns it into an *engineered* guarantee by checking every claim against the actual extracted PDF text before it's shown to the user.

Pipeline

1. LLM generates: `(claim)`, `(candidate_quote)`, `(claimed_page)`.

- Backend runs a fuzzy string match (e.g. `rapidfuzz.fuzz.partial_ratio`) of `candidate_quote` against the actual extracted text of `claimed_page` (and ± 1 adjacent page, in case of off-by-one page attribution).
- Match score determines a badge:
 - $\geq 90\%$ similarity $\rightarrow$  **Verified** (quote essentially found verbatim)
 - 60-90% similarity** $\rightarrow$  **Paraphrased** (idea is present, wording differs)
 - $< 60\%$ similarity $\rightarrow$  **Unsupported** (re-attempt retrieval once; if still unsupported, downgrade confidence and label clearly)
- Store the matched span's character offsets so the PDF viewer can highlight the exact matched text, not just jump to the page.
- Badge and match score are surfaced in the UI next to every claim, chat answer, and reviewer statement.

Output

```
json
{
  "claim": "Accuracy improved by 5%",
  "quote": "...improved from 82.3% to 87.4%",
  "page": 12,
  "match_score": 0.94,
  "status": "verified"
}
```

Demo value

This is the single most defensible "wow" moment: showing judges that citations aren't just generated, they're *checked* — live, on stage, against the actual PDF text.

Feature 5 — Grounded Chat

Users ask questions.

Example

Why did the authors choose transformers?

Response

Answer

...

Supporting Quote

...

Page

7

Confidence

High

Verification

☒ Verified

If evidence does not exist:

The uploaded paper does not provide enough evidence.

Every chat answer is routed through Feature 4B before being shown — no unverified answer is displayed without an explicit / badge.

Feature 6 — Citation Explorer

Extract references.

Fetch metadata.

Display

- Title
- Authors
- Year
- Citation Count
- Abstract

Also explain

Why was this paper cited?

Example

BERT
Foundational work
Introduced transformer pretraining.
Current paper builds upon it.

Reliability note: Semantic Scholar lookups can fail or rate-limit. Pre-cache metadata for the demo paper's reference list as a fallback so a live API hiccup doesn't break the demo.

Feature 7 — Research Family Tree

Visual graph

Transformer
↓
BERT
↓
RoBERTa
↓
Current Paper

Nodes should be clickable.

(Stretch feature — see reprioritized MVP stop points below.)

Feature 8 — Reviewer Mode

Acts as an academic reviewer.

Generates

Strengths

Weaknesses

Missing Experiments

Missing Baselines

Reproducibility Score

Example

Reproducibility

8.1 / 10

✓ Dataset

✓ Code

× Random Seed

✓ Hardware

× Statistical Significance

(Stretch feature — see reprioritized MVP stop points below.)

Feature 9 — Contradiction / Consistency Checker (*new*)

Description

Cross-checks claims made in different sections of the same paper — e.g., the abstract claims "state-of-the-art" results, but the results table shows the method losing to a baseline on 2 of 3 benchmarks.

Approach

1. Extract quantitative/qualitative claims from Abstract and Introduction.
2. Extract corresponding results from the Results section (table/text).
3. Ask the LLM to compare the two extracted sets and flag mismatches, with each flagged mismatch passed through Feature 4B for verification of both sides.
4. Display flags in Reviewer Mode as a "Consistency" sub-section.

Example

▲ Consistency Flag

Abstract claims: "achieves state-of-the-art performance"

Results (Page 9): method underperforms baseline X on 2 of 3 benchmarks

Both sides verified against source text.

(Stretch feature, builds on Reviewer Mode — only attempt after MVP 3 is stable.)

Feature 10 — Verified Summary Export (*new*)

Description

One-click export of the structured summary (contributions, methodology, results, limitations) as a shareable PDF/image card, with each claim's verification badge included.

Why

Cheap to build relative to its payoff — gives users something tangible to take away, and is a strong, clean note to end the demo on.

(Stretch feature — attempt only after core grounding + chat are solid.)

9. Dashboard

Upload Paper

Summary

Main Contribution

[//

Methodology

[//

Results

[//

Limitations

[//

PDF Viewer (with highlighted matched spans)

Evidence

Chat (each answer shows verification badge)

Citation Explorer

Reviewer Mode (+ Consistency flags)

Export Verified Summary

10. Functional Requirements

PDF Processing

- Upload PDF
- Parse pages
- Extract text
- Preserve page numbers

Summarization

Generate

- Contributions
- Methodology
- Results
- Limitations

Grounding & Verification

- Every generated statement must reference Page Number + Quote

- Every quote must be checked against actual page text via fuzzy matching
- Every claim/answer must carry a Verified / Paraphrased / Unsupported badge

Chat

Questions answered only from uploaded paper, routed through verification before display.

Citation Analysis

Extract references. Retrieve metadata (with cached fallback). Generate citation purpose.

Reviewer Mode

Analyze reproducibility, weaknesses, and (if time allows) cross-section consistency.

11. Non-Functional Requirements

Performance

Upload to summary: target < 20–35 seconds. Parallelize per-section LLM calls (contribution/methodology/results/limitations) rather than running them sequentially; relax the 20s target before sacrificing verification quality.

Accuracy

No claim is shown without a verification badge. "No unsupported claims" is an engineering target enforced by Feature 4B — not merely a prompt instruction. Claims that fail verification are explicitly labeled, not hidden or silently dropped.

Reliability

If Citation Explorer or Reviewer Mode fails (e.g. external API failure), wrap in try/except with a graceful UI fallback ("Citation data unavailable") so the rest of the dashboard still functions.

Usability

Entire core workflow (upload → summary → evidence) should require < 3 clicks.

12. Tech Stack

Frontend

- Streamlit (with `streamlit-pdf-viewer` or embedded `pdf.js` component for page-jump + highlight)

Backend

- Python

PDF Parsing

- PyMuPDF

LLM

- OpenAI GPT-4.1 / GPT-4o

Embeddings

- text-embedding-3-small

Vector Store

- FAISS

Verification

- `rapidfuzz` (fuzzy string matching for quote verification)

Citation Metadata

- Semantic Scholar API (with local cache fallback for demo reliability)

Visualization

- Plotly / PyVis / NetworkX
-

13. Folder Structure

```
paperlens/  
  
app.py  
  
parser/  
    pdf_parser.py  
    chunker.py  
  
ai/  
    summarizer.py  
    prompts.py  
  
rag/  
    embeddings.py  
    retriever.py  
  
verification/  
    verifier.py  
    fuzzy_match.py  
  
citations/  
    extractor.py  
    semantic.py  
    cache.py  
  
reviewer/  
    reviewer.py  
    consistency.py  
  
ui/  
    dashboard.py  
    export.py  
  
utils/  
  
tests/
```

14. Team Responsibilities

Member 1 — Document Intelligence

Responsible for

- PDF Upload
- Parsing

- Chunking
- Section Detection

Deliverable: Structured JSON document.

Member 2 — AI Summarization

Responsible for

- Contribution Summary
- Methodology
- Results
- Limitations
- Prompt Engineering

Deliverable: Evidence-linked summaries, ready to pass into Verification.

Member 3 — Grounded Chat & Verification (*expanded scope*)

Responsible for

- Embeddings
- FAISS
- Retrieval
- Question Answering
- **Evidence Verification Pipeline (Feature 4B)** — this is now the highest-priority individual deliverable on the team

Deliverable: Evidence-backed chat + a shared `verify_claim()` function used by Summarization, Chat, and Reviewer modules.

Member 4 — Citation Intelligence

Responsible for

- Reference Extraction
- Semantic Scholar API (+ local cache fallback)
- Citation Explorer
- Research Family Tree (stretch)

Deliverable: Interactive citation dashboard, resilient to API failure.

Member 5 — Frontend & Integration

Responsible for

- Streamlit Dashboard
- PDF Viewer with page-jump + highlight
- Verification badges in UI
- Navigation

- Final Integration
- Verified Summary Export (stretch)

Deliverable: Complete interactive interface.

15. Milestones

Milestone 1 (Hours 0-4)

- Repository setup
- UI skeleton
- PDF upload
- Shared JSON schema agreed across team (critical — avoids integration pain later)

Milestone 2 (Hours 4-8)

- Parsing
- Summaries (parallelized calls)
- Basic dashboard

Milestone 3 (Hours 8-12)

- **Evidence Verification Pipeline (Feature 4B) — build this before polishing anything else**
- Chat

Milestone 4 (Hours 12-16)

- PDF viewer page-jump + highlight
- Citation Explorer (+ cache fallback)

Milestone 5 (Hours 16-20)

- Reviewer Mode
- Consistency Checker (if ahead of schedule)
- Family Tree (if ahead of schedule)

Milestone 6 (Hours 20-24)

- Testing
 - UI polish
 - Verified Summary Export (if time allows)
 - Demo preparation
-

16. MVP Stop Points (*reprioritized around verification*)

MVP 1 (40%)

- Upload
- Summary

Demo Ready

MVP 2 (60%)

- Evidence Grounding
- **Evidence Verification badges (Feature 4B)**

Strong Submission — this is the point at which the core "proof, not just claims" pitch becomes real

MVP 3 (80%)

- Chat (verified)
- PDF page-jump + highlight
- Citation Explorer

Competitive Submission

MVP 4 (100%)

- Reviewer Mode
- Consistency Checker
- Family Tree
- Verified Summary Export
- UI Polish

Ideal Final Product

Cut order if behind schedule: Family Tree → Verified Summary Export → Consistency Checker → Reviewer Mode. Never cut the Verification Pipeline — it is the product's entire differentiation.

17. Demo Script (3 Minutes)

1. Upload a real research paper.
2. Display structured summary — point out the  /  /  badges next to each claim.
3. Click a claim, show the PDF jump to the exact page **with the matched quote highlighted**.
4. Deliberately ask a question the paper doesn't answer, and show the honest "not enough evidence" response — then ask one it does answer, showing a  Verified badge.
5. Explore the Research Family Tree (if built).
6. Open Reviewer Mode, show reproducibility score and any Consistency flags.
7. Export a Verified Summary Card.
8. Conclude with:

"PaperLens doesn't just summarize research — it teaches it with evidence that's actually verified, not just claimed."

